Sitezy: Hybrid Local-LLM Routing, Deep-Learning Post-Generation Optimisation, and Agentic Backend Automation for Multi-Tenant Website Generation

Ronit Sahoo, Atharva Lotankar, Manaswi Kabadi, Shakti Sankpal, and Kajal Joseph

Department of Information Technology, Vivekanand Education Society's Institute of Technology (VESIT), Mumbai, India
2023.ronit.sahoo@ves.ac.in, 2023.atharva.lotankar@ves.ac.in
2023.manaswi.kabadi@ves.ac.in, 2023.shakti.sankpal@ves.ac.in
kajal.jewani@ves.ac.in

Abstract. AI website builders send every generation request to a metered cloud language model and offer little measured optimisation once a site exists. We present *Sitezy*, a multi-tenant platform in which decisions with a bounded output space—block recommendation, SEO scoring, design auditing, conversion prediction—are made by small learned models, and only open-ended generation reaches a language model, served locally (Qwen3.5-4B) with a cloud model as an explicit exception. Executing the deployed code on 20 benchmark sites and 66 real published pages, we find that disabling the local model's reasoning mode is decisive: with it enabled, 8 to 20 of 20 generations exhaust the token budget; with it disabled, 59 of 60 are valid over three runs (McNemar $p < 10^{-4}$), at \$0.15 of GPU energy per 1,000 sites against \$13.43 for the 3.7× faster cloud model. On synthetic sessions, a GRU conversion model reaches 99.4% of the Bayes-optimal AUC, and a gated rewrite loop raises rubric-based SEO scores (a proxy, not search performance) without lowering any. Fault-injection and tenant-isolation tests found two access-control defects, since fixed. We also report negative results—a linear model fits the SEO rubric almost exactly, the local model leaves 32 of 59 forms empty until targeted regeneration fills them, and the anomaly detector does not transfer to real pages—and release all scripts and seeds.

Keywords: Local LLM · Inference routing · Constrained decoding · Multi-tenant SaaS · GRU · Anomaly detection · Ablation

1 Introduction

Building a small-business website couples layout, copywriting, search optimisation and a backend for customer input. AI website builders [1, 2] make layout tractable, but in a multi-tenant setting every content request is metered by a cloud model and sends tenant data off-platform; generated pages receive no

measured optimisation, although on-page attributes predict search visibility [14], sequence models capture visitor behaviour [15, 18], and anomaly detectors audit design without labelled failures [19, 21]; and a page stays inert until forms, storage and hosting are wired.

Primary contribution. We contribute a *local-first, task-typed* AI stack and its measured characterisation: bounded tasks are answered by small learned models (Tier 1) and only open-ended generation reaches a language model (Tier 2 local, Tier 3 cloud). The other components form one loop around it (Fig. 1): the language model generates a site, Tier-1 engines recommend, score and audit it, a gated rewrite step improves weak copy, backend agents publish it and provision form storage, and a tracker returns sessions to the conversion model. We contribute (i) the architecture, with exception-driven escalation and redaction before any cloud call (Sects. 3–6); (ii) a repeated decoding ablation, same-workload cloud baselines and a measured cost model (Sect. 7.1); (iii) an evaluation of the learned engines against stronger baselines, the deployed rules and the Bayes-optimal ceiling, on benchmarks and real pages (Sects. 7.2–7.6); and (iv) deployment fault injection, isolation tests and an end-to-end ablation (Sects. 7.7–8). All scripts and seeds are released.[1]

2 Related Work

Cost-aware LLM serving routes queries *among language models* [10]: FrugalGPT [8] cascades APIs and stops when a learned scorer accepts an answer, and RouteLLM [9] learns to send each query to a strong or a weak model. Sitezy instead routes by *task type*, so bounded tasks never reach a language model, and escalates on deterministic events rather than learned confidence. Grammar-constrained decoding [11] is designed to guarantee syntactically valid output; we show that for long outputs validity is decided by reasoning tokens competing for a fixed budget. Design2Code [12] benchmarks screenshot-to-code generation and WebGen-Bench [13] functional multi-file websites; we target what surrounds generation. Our engines build on gated recurrent units [15], which match LSTMs [16] with fewer parameters [17] and model sessions well [18]; on reconstruction-error [20, 21] and isolation-based [19] anomaly detection; and on entailment-based zero-shot classification [23] with MiniLM [24] feeding random forests [25]. Conversion effects require online controlled experiments [26] for validation, and shared-database isolation is a central multi-tenant risk [27]. Unlike these, and cloud AI builders whose routing is not documented, Sitezy routes by task type with deterministic escalation, runs local-first, constrains output, and adds four learned post-generation engines.

3 System Architecture

Sitezy (Fig. 1) comprises a React client (out of scope), a Node.js/Express API (AI router, collaboration, page versioning, deployment agents), a Python

[1] `https://github.com/shaktisankpal/SitePilot/tree/ronit/evaluation`

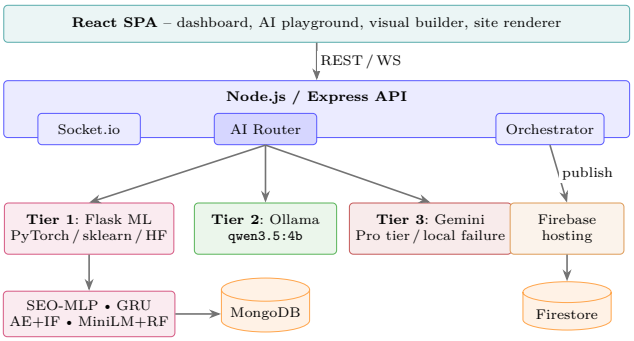


Fig. 1. Architecture. Bounded tasks go to Tier 1, open-ended generation to Tier 2 (local); Tier 3 (cloud) is used only on the Pro tier or a local failure. All stores are tenant-scoped.

Table 1. Routing and escalation as implemented. Apart from form repair, no condition re-prompts a model.

Condition	Action	Cloud
Bounded task (recommend, score, audit, predict)	Tier-1 learned model (rule if artifact missing)	never
Open-ended generation, Basic tier	Tier 2: `qwen3.5:4b`, JSON grammar, no reasoning	no
Open-ended generation, Pro tier selected	Tier 3 directly	yes
Tier-2 runtime error (unreachable, timeout)	Tier 3 on the same prompt	yes
Output unparseable or fails the layout schema	deterministic template layout	no
Form with <3 valid fields (Sect. 7.5)	Tier 2 regenerates that form only (fixed fields if it fails)	no

Flask service with the four Tier-1 engines, and an Ollama server running `qwen3.5:4b` [5, 6] (4.7 B parameters, Q4_K_M, 8,192-token context, 6.1 GB of VRAM); Tier 3 is `gemini-3-flash-preview` [3, 7]. MongoDB holds 16 collections and Firestore the per-tenant form submissions; sites are served by the platform renderer or exported to Firebase Hosting. Each Tier-1 endpoint degrades to a deterministic rule if its model artifact is missing.

4 Generation Tier: Routing, Decoding, and Data Handling

Routing. The router is rule-based, with no learned confidence threshold (Table 1): escalation follows observable events (a tenant's choice or a local runtime exception), since a validity check is exact and cheap while a threshold would need calibration data a new tenant lacks, and every assignment stays auditable. Invalid output from either model is replaced by a deterministic template layout, never by a second generation call, which bounds cloud spend.

Table 2. Synthetic training data. $\tilde{x}_k$: min–max normalised features; c, e, s, d, b: CTA click, early CTA, form start, form done, bounce; n: visible sections.

Engine	Inputs	Label	Size
SEO scorer	15 uniform on-page features (title 0–90 chars, meta 0–220, keyword density 0–6%, words 20–800, readability 0–20, alt coverage 0–1, link/heading counts, binary flags)	$100\sum_k w_k\tilde{x}_k + \mathcal{N}(0, 3^2)$, clipped to $[0, 100]$; hand-set w (heading hierarchy .16, keyword density .13, alt .12, ...)	20k/5k per seed
Conversion GRU	Navbar, Hero, 2–6 shuffled sections, Footer; bounce .35 truncates; CTA click .55 if the CTA is in the first four sections, else .30; form start .5, completion .6; dwell $\mathcal{N}(12, 4)$ s (content) or $\mathcal{N}(6, 4)$ s	$p = .06 + .30c + .10e + .12s + .30d + .08(1-b) + \min(.1, .015n)$, clipped; $y \sim \text{Bern}(p)$	10k/4k per seed
Design auditor	16 builder templates as $\mathbf{f} \in [0, 1]^{20}$, each with 130 Gaussian jitters ($\sigma = .04$)	none (clean designs only)	2,096

Constrained decoding. Tier-2 calls set `format:json`, a JSON grammar [11], and `think:false`, which disables the reasoning mode, with a 5,000-token budget, an 8,192-token context and temperature 0.1. A three-page site needs about 4,000 output tokens, so reasoning tokens compete with the site for the same budget (Sect. 7.1).

Data handling. On the cloud path, owner-authored content for publication (business description, tone, audience, brand facts, page copy) is sent as is; user- and visitor-submitted data (form submissions summarised by the analytics assistant, chatbot and help questions) is redacted first: personal fields (name, e-mail, phone, address, contact) are masked and e-mail addresses and phone numbers removed from free text, keeping dates, prices, quantities and product choices. The local model receives unredacted data, which stays on the host.

5 Post-Generation Engines (Tier 1)

Training data. No public dataset pairs page structure with search outcomes, visitor sequences, or design-quality labels, so the engines are trained on procedurally generated data (Table 2); Sect. 9 discusses the consequences.

SEO scorer and gated rewrite loop. An extractor turns a page into 15 features $\mathbf{x}$, and an MLP outputs $s = 100\,\sigma(W_3\phi(W_2\phi(W_1\mathbf{x} + b_1) + b_2) + b_3)$ with hidden widths 64 and 32, ReLU ϕ and dropout 0.2 (deployed: Adam, 160 epochs, MSE). Improvement has three stages. (1) While $s < 80$, up to three of the weakest text-fixable factors go to Tier 2 (JSON, no reasoning, temperature 0.4), which proposes patches to *existing* text fields; the page is re-scored and the patch set is kept only if $s' > s$, otherwise it is discarded and the stage stops ($\leq$3 iterations). (2) A deterministic step adds the target keyword once to a body field, under the same gate. (3) A deterministic step writes a meta description and JSON-LD when those factors are weak; it is not gated, so we measure it separately.

Conversion GRU. A visitor session is padded to 20 steps of 12 features (7-way section-type one-hot, dwell time, scroll depth, CTA click, form start, form completion). Two GRU layers [15] (64 and 32 units; state $h_t = (1 - z_t) \odot$

$h_{t-1} + z_t \odot \tilde{h}_t$ with update gate z_t) feed a dense layer and a sigmoid giving the conversion probability p (BCE, Adam, 14 epochs). Before a site has traffic, a session is synthesised from its section order. Suggestions are counterfactual: the page is re-scored with a section moved and the *predicted* change Δp is reported; since p depends on section order, a bag-of-features model cannot provide this.

Design auditor. A page is encoded as $\mathbf{f} \in [0,1]^{20}$ (section and CTA counts, heading hierarchy, WCAG contrast [22], font and colour diversity, alt coverage, duplicate-heading and empty-section ratios, structural order). A deterministic rubric of eleven checks with severity penalties (high 22, medium 11, low 5) yields the health score shown to users. A symmetric autoencoder (20–14–8–4, mirrored) and an Isolation Forest [19] (200 trees), both trained only on clean designs, give standardised anomaly scores whose maximum is shown as an informational signal.

Block recommender. A MiniLM cross-encoder trained for entailment [24] (`nli-MiniLM2-L6-H768`) classifies a prompt zero-shot [23] over four label sets (13 business types, 11 content types, 9 goals, 10 audiences), choosing the label with the highest entailment probability for "This website is about *label*". The labels feed one random forest [25] per block type (12 blocks); blocks above a business-specific threshold (0.45–0.60) are selected, keyword matches add blocks, and the set is padded to at least four. Tier 2 then fills this skeleton with copy.

6 Backend Automation

Publishing copies a site's pages into an immutable, numbered snapshot served on a tenant subdomain or custom domain; rollback restores a snapshot (11 tenants published 166 versions of 73 websites during the logging period). Every home page carries a form with five to eight typed fields (`text`, `email`, `tel`, `number`, `select`, `textarea`, `date`), validated and, if empty, regenerated (Sect. 7.5), which posts scalar values of at most 5,000 characters to the site's tenant-scoped Firestore path, so no tenant-written backend code is needed. An optional agentic pipeline exports a site to Firebase Hosting: behind a per-tenant circuit breaker (open after five failures in five minutes), a *FirebaseAgent* deploys pages and assets, applies security rules and provisions the form backend; after a failure a *DiagnosingAgent* classifies the error (quota, permission, network, credentials, isolation breach, timeout, storage), re-checks isolation, rules and quota, and assigns a severity: medium (network, timeout, unclassified) is retried once after a 2 s back-off, critical (permission, credentials, isolation) and high (quota, storage) are not.

7 Experimental Evaluation

Setup and protocol. All runs used one laptop: NVIDIA RTX 4060 Laptop GPU (8 GB), AMD Ryzen 7 7840HS, 16 GB RAM; Ollama 0.21, Python 3.13, PyTorch 2.11 (CPU), scikit-learn 1.7, Transformers 5.6, Node.js 22. Trained engines are evaluated over five seeds with freshly generated, seed-disjoint training

Table 3. Generation on the 20 production prompts; rows with 60 (40) generations pool three (two) independent runs. Valid = passed the layout schema (Wilson 95% interval); truncated = stopped at the 5,000-token budget; medians; output tokens include reasoning tokens.

Tier	Decoding	Valid	Truncated	Median s	Output tok.
Local	reasoning on, free-form	0/20 [0, 16]	20	99.1	5,000
Local	reasoning off, free-form	20/20 [84, 100]	0	78.3	3,919
Local	reasoning on, JSON grammar	29/60 [36, 61]	31	101.8	5,000
Local	**reasoning off, JSON grammar (deployed)**	**59/60 [91, 100]**	**0**	**77.4**	**3,932**
Cloud	Gemini 3 Flash (production path)	40/40 [91, 100]	0	20.8	2,866 + 1,270 reasoning
Cloud	Gemini 3 Flash, matched constraints	12/20 [39, 78]	8	19.2	2,729 + 1,318 reasoning

and test sets (means, 95% t-intervals, paired t-tests across seeds); proportions carry Wilson intervals [29], and paired comparisons on the same prompts use the exact McNemar test [28]. Generation experiments call the production prompt, system message and options verbatim on 20 businesses (coffee shop to co-working space), requesting all seven section types as when the recommender is unavailable, and judge validity with the production extraction and layout-schema check.

7.1 Generation Tier

Table 3 reports a paired four-way decoding ablation and a same-workload cloud baseline: the production cloud path (no system message, JSON mode or token cap) and a matched variant with the local call's system message, JSON mode, temperature and 5,000-token budget. The deployed configuration and reasoning with grammar were each run three times and the production cloud path twice (a third cloud run, stopped by the API quota, is excluded).

The reasoning mode decides validity. With it enabled and no grammar, every generation exhausts the budget before the JSON closes; the grammar recovers 12, 7 and 10 of 20 in the three runs (first run vs. free-form: McNemar p=0.0005), the rest truncating. With reasoning disabled, 79 of 80 generations are valid with or without the grammar, and faster (median 77.4 s vs. 99–102 s); the one failure closed a brace too early, so Ollama's JSON mode does not guarantee well-formed output and the schema check and template fallback remain necessary. `think:false` is thus decisive (vs. free-form $p=1.9 \times 10^{-6}$; vs. reasoning with grammar p=0.008, 0.002 and 0.002 per run), and latency is stable across runs (medians 79.0, 77.1, 76.8 s; Friedman p=0.70). Under the same constraints the cloud model fails alike: on 8 of 20 sites its reasoning tokens exhausted the 5,000-token budget (McNemar p=0.008 vs. the production path), so part of its advantage comes from an uncapped budget.

Cost and field usage. The cloud tier is billed $p_{\text{in}} t_{\text{in}} + p_{\text{out}}(t_{\text{out}} + t_{\text{reason}})$ per site at list price ($0.50/$3.00 per million input/output tokens [4]): $13.43 per 1,000 sites over both runs ($13.72 and $13.15; per site $11.36–15.68); failed requests (one transport error, retried) return no tokens and are not billed. The local tier's marginal cost is energy, $P\,\bar{t}\,c_e$, with board power P=60 W (`nvidia-smi`), $\bar{t}$=77.4 s and an assumed c_e=$0.12/kWh: 1.3 Wh or $0.15 per 1,000 sites, about 1.2% of the cloud fee, at 3.7× the latency (the cloud median moved from 26.0

Table 4. SEO scorer vs. baselines (5 seeds, 5,000 test pages each; mean ± 95% CI). R^2 and MAE against the noise-free rubric; latency per page.

Model	R^2	MAE (0–100)	Latency (ms)
Linear regression	1.000 ± 0.000	0.07 ± 0.02	0.04
Histogram gradient boosting	0.991 ± 0.000	0.90 ± 0.02	3.2
MLP, tuned (dropout .05, 900 epochs)	0.977 ± 0.009	1.62 ± 0.36	0.09
Random forest (200 trees)	0.956 ± 0.002	2.01 ± 0.04	8.2
MLP, deployed configuration	0.738 ± 0.105	5.93 ± 1.35	0.09

to 18.8 s between runs). This excludes rest-of-system power, GPU amortisation, operations and cloud free-tier allowances; "near-zero marginal cost" refers only to this energy figure. In the field, 19 tenants logged 251 generation requests (February–September 2026): before local serving existed all went to the cloud and 38% failed, mostly on rate limits and credentials; afterwards 56 of 153 (36.6%) were served locally, 53 successfully. The log covers site generation only, omits why a request went to the cloud, and reflects development-period, not steady-state, traffic.

7.2 SEO Engine

All SEO figures in this paper are scores against our rubric (Table 2), a proxy built from published ranking factors [14]; none is a measured ranking, impression or click-through rate. Because the labels are a linear function of the features plus noise (Table 2), linear regression recovers the noise-free rubric to within estimation error (R^2=0.99995, MAE 0.07; 0.943 against the noisy labels, the noise ceiling); the deployed MLP configuration under-fits (Table 4). On rubric-derived labels the learned scorer therefore adds no accuracy; its value is as a trainable component to be re-fitted to real search outcomes (Sect. 9).

Gated rewrite loop. On the 60 pages of the 20 sites (51 below the threshold), the loop raised the deployed score from 56.4 ± 4.8 to 65.2 ± 4.4 and the rubric score from 63.3 ± 5.3 to 72.7 ± 4.7; pages at or above the deployed scorer's threshold rose from 9 to 20 and none ended lower. The deterministic metadata step contributed more than the LLM (5.3 ± 0.7 vs. 3.1 ± 1.5 points; keyword step 0.3 ± 0.4). The gate makes the LLM stage safe: of 79 calls, 35 proposals were accepted (none lowered the rubric score), 16 had no applicable patch and 28 were rejected, 26 of which would have lowered the score (mean -2.0, worst -10.7). A rewrite took 7.5 s; scoring takes 0.32 ms.

7.3 Conversion Model

On 4,000 test sessions per seed (38% positive), the Bayes-optimal AUC (ranking by the true p) is 0.778 ± 0.006. The deployed GRU reaches 0.773 ± 0.006, 99.4% of it, with 25,473 parameters, 24% fewer than an equally accurate LSTM (0.774 ± 0.007, 33,601; p=0.24); mean-pooled MLP and logistic models (449 and 13 parameters) are equivalent (0.774 ± 0.008, 0.769 ± 0.007), and all learned models beat the deployed rule fallback (0.709 ± 0.008) by 0.061–0.065 AUC

(GRU: p=6 × 10^{-5}); F1 is 0.641–0.645 for all learned models and the Bayes ranking vs. 0.588 for the rule. Sequence modelling brings no accuracy gain on these aggregate-driven labels; we keep the GRU because only it makes predictions depend on section order, which counterfactual suggestions need. Moving a buried CTA after the hero raised the *predicted*—not measured—conversion by 0.061 ± 0.027 on all 15 of the 16 templates that have one; suggestions take 8 ms.

7.4 Design Auditor

We injected section-level flaws into the 16 templates, none of which triggers a rubric check: ten *catalogued* categories matching rubric checks (missing H1: high severity; multiple H1, no H2, buried CTA, empty sections, missing alt text: medium; no footer, too many fonts, duplicate headings, unusual order: low) and five *out-of-catalogue* anomalies (section bloat, gallery overload, sparse page, colour chaos, wall of text). Thresholds were set on even-numbered templates and results reported on odd-numbered ones (per seed: 160 clean, 160 catalogued, 80 out-of-catalogue pages). The rubric is perfect on catalogued flaws by construction but catches only 7.5% of out-of-catalogue anomalies (AUC 0.537); AE+IF (max) keeps catalogued F1 at 0.999 ± 0.003 and catches 79.7 ± 0.7% of them (AUC 0.897 ± 0.015; all but "wall of text", which the length feature cannot represent) at a 0.3% clean false-positive rate. The autoencoder alone matches it (F1 0.992 ± 0.007, recall 79.7 ± 0.7%, AUC 0.899 ± 0.010); the Isolation Forest alone is weaker (0.805 ± 0.031, 70.5 ± 7.5%, 0.860 ± 0.028). Auditing a page takes 6.7 ms.

7.5 Recommender and Form Schemas

No expert annotations of block sets exist, so we evaluate the zero-shot stage that feeds the forests on 52 prompts we wrote ourselves (four per business type). Entailment scoring identifies the business type in 43 (82.7%, Wilson [70.3, 90.6]; chance 7.7%); an earlier version that scored the neutral class managed 26 (50.0%; McNemar p=0.0009), and the fix changes the selected blocks for 98% of prompts. A recommendation takes 1.37 s on the CPU, almost all in 43 sequential cross-encoder passes; batching them cuts this to 0.37 s. The evaluation exposed an integration defect: the backend silently dropped 63 of the 152 blocks recommended for the 20 benchmark businesses (services, pricing, FAQ, team, statistics, newsletter, blog) and never requested the order form; both are fixed (closest section type plus a hint; order form always requested) and measured in Sect. 7.8.

Schema validity does not imply complete content. Gemini filled the required form on all 20 sites, but the local model left the field list empty on 12, 10 and 10 of the 20, 19 and 20 valid sites of its three runs, and post-processing added none. All 191 local and 268 cloud fields conformed to the schema; e-mail and phone fields always had the matching type, `select` fields offered 4.3–4.8 options, and 62% of local vs. 79% of cloud fields were required; 29% of local field names (first run) were copied from the prompt's examples (cloud: 7%). We therefore added *field validation and targeted regeneration*: a form with fewer than three

valid fields (name, label, allowed type, $\geq$2 `select` options, unique names) is regenerated alone by a short Tier-2 call for 5–8 business-specific fields, with fixed fields as fallback. On the 59 valid sites all 32 empty forms were regenerated by the model (fallback unused), giving 59/59 filled forms with 6–8 valid fields (all 220 conformant) and 7% example reuse, at 8.7 s per regeneration (4.8 s per site).

7.6 Real Tenant Pages

We ran the deployed engines on all 66 published pages of 28 tenant websites (read-only; only aggregate metrics stored). Rubric scores were 68.1 ± 4.2 (deployed scorer 60.6 ± 3.8), with 58 pages below the threshold; the loop raised them to 75.1 ± 3.7 and the pages at or above the threshold from 8 to 15, lowering none (33 proposals accepted, 41 rejected, 37 of them harmful). The rubric flagged 19 pages, mostly for a missing H1 or a buried CTA (18 each). The anomaly detector flagged 63 of 66, including 44 the rubric found nothing wrong with: trained on 16 home-page templates, it flags page type and size (all 24 pages with at most four sections, 57 of 58 inner pages) rather than design quality, and must be retrained before it transfers. Counterfactual reordering predicted a gain above 0.01 for 13 of the 57 pages that admit a move. All 30 forms had fields.

7.7 Deployment Pipeline

We ran the agentic pipeline against a Firebase test project under fresh tenant IDs (all test data deleted afterwards). All 20 benchmark sites deployed on the first attempt (median 0.29 s), so we injected six error classes and an unclassified error into the deploy call, once (transient) or on every attempt (persistent), three times each, under three retry policies. With the DiagnosingAgent, all 42 faults were classified correctly and the 9 transient medium-severity ones recovered; without retries none of the 21 transient faults recovered; blind retry recovered all 21 but spent 21 extra attempts on persistent faults (agent: 9), retried 12 permission and credential failures and took twice as long (99 vs. 51 s). The agent retries only what is safe, at the cost of 6 recoverable quota and storage faults; after five failures the circuit breaker rejected further deploys. These are fault-injection results on a test project, not production failure rates.

7.8 End-to-End Ablation

We ran the pipeline end to end on the same 20 sites, changing one factor per arm (Table 5; paired t-tests over sites). An independent re-run reproduced the full system (72.0 ± 4.8; p=0.10, 31 of 60 pages identical, none regressed). Removing the SEO loop costs 9.4 rubric points, the largest local effect; removing form repair saves 6 s per site but leaves 12 of 20 forms empty. Reasoning replaces 8 of 20 sites with the template and lowers the score by 3.9 points (not significant, p=0.07). Without the gate, 35 of 84 accepted edits lowered the score and 3 pages ended

Table 5. End-to-end ablation on the 20 benchmark sites (60 pages), one factor changed per arm. SEO: final rubric score, mean ± 95% CI over pages (text p-values are paired over sites; the re-run is in the text); forms filled: forms with ≥3 valid fields after repair; regressed: pages below their start; time and cost per site: generation (median over runs), rewriting, LLM scoring and form repair, as marginal GPU energy plus API fees.

Arm	Model layout	Forms filled	SEO	Regressed	Time (s)	$/1k sites
Full system (all seven sections)	20/20	20/20	72.7 ± 4.7	0/60	113	0.23
No form repair	20/20	8/20	72.7 ± 4.7	0/60	107	0.21
Reasoning enabled	12/20	12/20	68.8 ± 5.1	0/60	123	0.25
No acceptance gate	20/20	20/20	71.9 ± 4.7	3/60	131	0.26
Rule scorer (no learned model)	20/20	20/20	71.8 ± 4.5	0/60	107	0.21
LLM scorer (Tier 2 instead of Tier 1)	20/20	20/20	71.3 ± 4.8	0/60	120	0.24
Block recommender chooses sections	18/20	18/20	77.9 ± 4.6	0/60	95	0.19
No SEO loop	20/20	20/20	63.3 ± 5.3	0/60	83	0.17
Cloud generation	20/20	20/20	78.0 ± 4.2	0/60	44	13.48

below their start, yet the mean fell only 0.8 points (p=0.12): the gate prevents regressions rather than raising the average. *Routing* the scoring step matters: the rubric as scorer is indistinguishable from the learned model (−1.0, p=0.08), but the Tier-2 model loses 1.5 points (p=0.001) and adds 7 s per site, rejecting 51 of 67 proposals. With the recommender choosing sections, home pages average 8.4 sections (fallbacks included) instead of 7 and score 77.9 (+5.2, p=0.06), on a par with cloud generation and 18 s faster; two longer sites truncated and are scored as served, with the template (+7.5, p=0.004 on the other 18). Cloud generation scores 78.0 (p=0.005) at 44 s per site but costs $13.48 ($13.43 of API fees plus $0.05 of rewriting energy) instead of $0.23 ($0.15 of generation energy plus rewriting and repair) per 1,000 sites, and sends all generation data off-host. The top reorder suggestion changes nothing measurable on generated pages: all 20 home pages share the prompt-mandated order and get the same suggestion (predicted +0.062), with no design or SEO rubric gain (−0.2); its value lies with hand-built pages (Sect. 7.6). The anomaly detector is informational and cannot change a page.

8 Security and Multi-Tenancy

Every tenant-owned document carries a `tenantId`. An authentication chain verifies the JWT (seven-day expiry), rejects tokens whose tenant differs from the user's, checks that the tenant is active and injects a tenant filter into queries; four roles gate mutating routes, passwords use bcrypt (12 rounds), and the API is rate-limited. Public endpoints serve only published content. A static audit of all 163 database call sites found 118 with the tenant filter in the call; the other 45 are inherently cross-tenant (tenant or domain resolution, global uniqueness, administration and billing: 31), scoped through a tenant-filtered parent (2), the published-pages chatbot (1), or offline scripts unreachable over HTTP (11).

Dynamic isolation tests. On an empty in-memory database we registered two tenants through the API and sent tenant B's owner token with tenant A's identifiers to all 39 route–method pairs that take a resource identifier; a request counted as blocked only if it returned 401/403/404, left A's documents byte-

identical and echoed none of A's identifiers (12 positive controls with A's token succeeded). All 39 were blocked; 10 list endpoints exposed nothing of A; 7 tampered tokens (e.g. `alg:none`, wrong secret, a foreign tenant claim) were rejected, as was an editor on 8 owner/admin routes even with a forged `OWNER` claim; public endpoints served no drafts. Two defects were found, fixed and re-tested (93/93 checks pass): a domain could be linked to another tenant's website, and the deployment route lacked a role check. These are functional authorisation tests, not a penetration test; payment and LLM routes were excluded.

9 Limitations and Future Work

- **Synthetic labels, predicted lift.** The SEO labels are a linear rubric, so the scorer is not a ranking model until re-fitted to real search outcomes; sessions are simulated and flaws injected. All Δp are predictions; the tracker logged only six engagement events, so online controlled experiments [26] remain necessary.
- **Transfer and completeness.** The anomaly detector must be retrained on real page types (Sect. 7.6); of the generated content, only structure and form fields are validated.
- **Scope of evidence.** One laptop GPU; 251 development-period requests without escalation reasons; deployments on a test project with injected faults; no expert labels for block-level recommender accuracy, and author-written test prompts; no human evaluation of copy and no regression suite beyond the released scripts; no penetration test or external security audit.

10 Conclusion

Most decisions an AI website builder makes do not need a language model. Small learned models answer bounded questions in milliseconds; a 4-billion-parameter local model produces schema-valid sites in 59 of 60 cases once its reasoning mode is disabled, at about 1% of the cloud's marginal cost, with targeted regeneration repairing the forms it leaves empty; and cloud calls become an explicit, auditable exception. Executing the deployed code also showed where learning does not yet help: a linear model fits the rubric-derived SEO labels, sequence modelling adds no accuracy on synthetic sessions, and the design anomaly detector does not transfer to real pages. The most valuable next step is therefore real data—search outcomes, visitor sessions and controlled experiments—to which the Tier-1 engines can be re-fitted.

References

1. Wix.com Ltd.: Wix ADI. `https://www.wix.com/adi` (2024)
2. Webflow, Inc.: Webflow. `https://webflow.com` (2024)
3. Gemini Team, Google: Gemini: a family of highly capable multimodal models. arXiv:2312.11805 (2023)

4. Google: Gemini Developer API pricing. `https://ai.google.dev/gemini-api/docs/pricing` (accessed October 2026)
5. Yang, A., et al. (Qwen Team): Qwen3 technical report. arXiv:2505.09388 (2025)
6. Ollama: qwen3.5 model library entry. `https://ollama.com/library/qwen3.5` (accessed October 2026)
7. Google: Gemini API models. `https://ai.google.dev/gemini-api/docs/models` (accessed October 2026)
8. Chen, L., Zaharia, M., Zou, J.: FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv:2305.05176 (2023)
9. Ong, I., et al.: RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665 (2024)
10. Varangot-Reille, C., et al.: Doing more with less: a survey on routing strategies for resource optimisation in large language model-based systems. J. Artif. Intell. Res. **86**, Article 14 (2026)
11. Willard, B.T., Louf, R.: Efficient guided generation for large language models. arXiv:2307.09702 (2023)
12. Si, C., et al.: Design2Code: How far are we from automating front-end engineering? arXiv:2403.03163 (2024)
13. Lu, Z., et al.: WebGen-Bench: evaluating LLMs on generating interactive and functional websites from scratch. In: NeurIPS (2025)
14. Ziakis, C., et al.: Important factors for improving Google search rank. Future Internet **11**(2), 32 (2019)
15. Cho, K., et al.: Learning phrase representations using RNN encoder-decoder for statistical machine translation. In: EMNLP, pp. 1724–1734 (2014)
16. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Computation **9**(8), 1735–1780 (1997)
17. Chung, J., et al.: Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv:1412.3555 (2014)
18. Hidasi, B., et al.: Session-based recommendations with recurrent neural networks. In: ICLR (2016)
19. Liu, F.T., Ting, K.M., Zhou, Z.-H.: Isolation forest. In: IEEE ICDM, pp. 413–422 (2008)
20. Sakurada, M., Yairi, T.: Anomaly detection using autoencoders with nonlinear dimensionality reduction. In: Proc. MLSDA Workshop, pp. 4–11. ACM (2014)
21. Zong, B., et al.: Deep autoencoding Gaussian mixture model for unsupervised anomaly detection. In: ICLR (2018)
22. W3C: Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation (2018)
23. Yin, W., Hay, J., Roth, D.: Benchmarking zero-shot text classification: datasets, evaluation and entailment approach. In: EMNLP-IJCNLP, pp. 3914–3923 (2019)
24. Wang, W., et al.: MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers. In: NeurIPS (2020)
25. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001)
26. Kohavi, R., et al.: Controlled experiments on the web: survey and practical guide. Data Mining and Knowledge Discovery **18**(1), 140–181 (2009)
27. Bezemer, C.-P., Zaidman, A.: Multi-tenant SaaS applications: maintenance dream or nightmare? In: Proc. IWPSE-EVOL, pp. 88–92. ACM (2010)
28. McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947)
29. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. J. Am. Stat. Assoc. **22**(158), 209–212 (1927)